

Współczesne aplikacje webowe

Witryna internetowa czasopisma naukowego

Patrycja Janik, Patryk Weklicz, Kacper Śliwka

25 marca 2026

Spis treści

1	Wprowadzenie	3
1.1	Opis projektu	3
1.2	Funkcjonalności	3
1.3	Struktura witryny	4
1.3.1	Panel logowania	5
1.3.2	Panel użytkownika	5
1.3.3	Panel administratora	5
2	Implementacja	6
2.1	Architektura i Logika Serwerowa (Backend)	6
2.2	Interfejs Użytkownika (Frontend)	7
2.3	Diagram klas UML	8
2.4	Logika aplikacji	9
2.5	Logowanie	9

1 Wprowadzenie

1.1 Opis projektu

Celem projektu jest stworzenie webowego systemu do zarządzania i udostępniania artykułów naukowych w formacie PDF. Witryna ma służyć czytelnikom do przeglądania zasobów oraz administratorom do sprawnego zarządzania treścią.

1.2 Funkcjonalności

Strona będzie posiadać niżej wypisane funkcjonalności:

- System automatycznie generuje nazwę pliku artykułu zgodnie ze schematem X-YY-NNN.pdf:
 - X – oznaczenie kategorii (M – matematyka, I – informatyka, D – dydaktyka, P – popularyzacja nauki),
 - YY – dwucyfrowy rok publikacji,
 - NNN – kolejny numer artykułu w danym roku i kategorii.
- Pliki artykułów mogą być przesyłane poprzez wybór z systemu plików lub metodą „przeciągnij i upuść” (drag and drop).
- System automatycznie przypisuje artykuł do odpowiednich sekcji („Najnowsze artykuły” oraz „Roczniki”) na podstawie daty publikacji. W przypadku braku artykułów w bieżącym roku, system automatycznie wyświetla artykuły z roku poprzedniego.
- Artykuły są grupowane według kategorii (matematyka, informatyka, dydaktyka, popularyzacja nauki) oraz sortowane malejąco według daty publikacji.

- Każdy artykuł zawiera:
 - tytuł,
 - autorów,
 - zakres stron,
 - datę publikacji,
 - kategorię,
 - plik PDF,
 - opcjonalny plik dodatkowy.
- System zapewnia walidację przesyłanych plików (typ PDF oraz ograniczenie rozmiaru).

1.3 Struktura witryny

Strona zawiera następujące zakładki:

- O czasopiśmie
- Najnowsze artykuły
- Roczniki
- Redakcja
- Informacje dla autorów
- Proces recenzji
- Recenzenci
- Kontakt

1.3.1 Panel logowania

System posiada mechanizm logowania wyłącznie dla administratora. Zwykły użytkownik nie posiada konta i korzysta z systemu anonimowo. W systemie wyróżniono następujące role:

- Zwykły użytkownik (anonimowy)
- Administrator

1.3.2 Panel użytkownika

Czytelnik ma dostęp wyłącznie do funkcji przeglądania:

- przeglądanie artykułów według kategorii i roczników,
- wyszukiwanie artykułów (autor, tytuł, słowa kluczowe),
- otwieranie plików PDF,
- przeglądanie informacji o czasopiśmie.

1.3.3 Panel administratora

Administrator posiada pełne uprawnienia:

- zarządzanie wszystkimi artykułami,
- zatwierdzanie lub odrzucanie artykułów,
- edycja i usuwanie artykułów,
- przeglądanie statystyk.

2 Implementacja

W celu realizacji witryny czasopisma naukowego zastosowana zostanie architektura typu klient-serwer. Warstwa prezentacji (frontend) oraz warstwa logiki biznesowej (backend) zostanie wyraźnie odseparowana, a komunikacja między nimi odbywać się będzie za pośrednictwem bezstanowego interfejsu REST API.

2.1 Architektura i Logika Serwerowa (Backend)

Logika serwerowa zostanie zaimplementowana z wykorzystaniem platformy .NET oraz frameworka ASP.NET Core Web API, co opiera się na wzorcu projektowym MVC (Model-View-Controller) w kontekście udostępniania zasobów przez kontrolery API.

- **Architektura systemu i wzorce projektowe:** System zostanie zaprojektowany zgodnie z paradygmatem **Clean Architecture** (Czystej Architektury), co zagwarantuje ścisłą separację zagadnień (Separation of Concerns) oraz wysoką testowalność. Rozwiązanie logicznie podzielono na cztery warstwy: *Domain* (rdzenne modele biznesowe), *Application* (logika przypadków użycia), *Infrastructure* (implementacja dostępu do danych i usług zewnętrznych) oraz *WebApplication* (punkty wejściowe API). Dodatkowo, w celu zmniejszenia powiązań między komponentami oraz uporządkowania przepływu żądań, w warstwie aplikacji zastosowany zostanie wzorec projektowy **Mediatora**.
- **Technologia bazowa:** C# oraz ASP.NET Core zapewniają wysoką wydajność, bezpieczeństwo przetwarzania danych oraz wbudowane mechanizmy wstrzykiwania zależności (Dependency Injection).
- **Zarządzanie danymi:** Jako system zarządzania relacyjną bazą danych wybrano SQLite. Jest to lekka, oparta na pojedynczym pliku baza danych, która nie wymaga konfiguracji i utrzymywania zewnętrznego serwera, co znacząco upraszcza proces wdrożenia oraz testowania aplikacji. Dostęp do danych realizowany jest za pomocą narzędzia typu ORM (Object-Relational Mapping) – Entity Framework Core. Baza danych przechowywać będzie zdefiniowane obiekty artykułów, zawierające m.in. tytuł, autorów, datę publikacji oraz przypisaną kategorię.

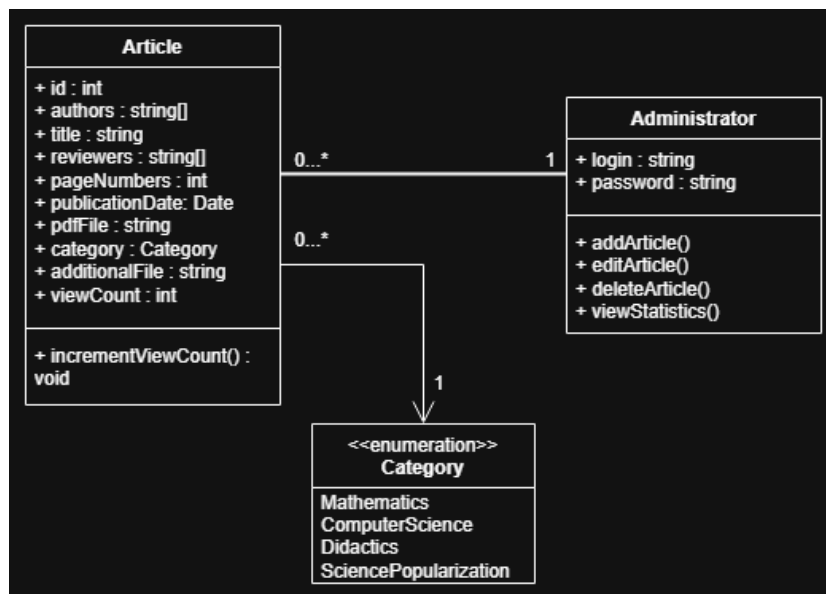
- **Uwierzytelnianie (Auth):** Zgodnie z wymaganiami, system będzie rozróżniał użytkownika oraz administratora. Zamiast polegać na certyfikatach logowania, wdrożony zostanie nowoczesny mechanizm oparty na standardzie JWT (JSON Web Tokens). Zapewni to bezpieczny dostęp do chronionego panelu administracyjnego.
- **Obsługa plików:** Artykuły udostępniane będą wyłącznie w formie PDF. W backendzie zaimplementowany zostanie dedykowany serwis plików, który ma na celu weryfikować ich typ oraz nadawać automatycznie nową nazwę zgodnie ze ścisłym schematem czasopisma (X-YY-NNN.pdf), bez względu na to, jak plik nazwał administrator przed przesłaniem. Przesyłane pliki PDF są przechowywane poza katalogiem publicznym oraz pod bezpiecznymi nazwami, aby uniemożliwić nieautoryzowany dostęp i wykonanie złośliwego kodu.

2.2 Interfejs Użytkownika (Frontend)

Aplikacja kliencka zostanie zrealizowana w architekturze SPA (Single Page Application) przy użyciu podejścia opartego na komponentach (COP).

- **Biblioteka główna:** Interfejs zbudowany będzie przy użyciu biblioteki React oraz języka TypeScript. Statyczne typowanie w TypeScriptie znacząco redukuje błędy i ułatwia zarządzanie przepływem danych między komponentami.
- **Podgląd dokumentów:** Do natywnego renderowania dokumentów w oknie przeglądarki zastosowana zostanie biblioteka `@pdf-viewer/react` lub inny jej odpowiednik. Jest to kluczowe dla realizacji funkcji płynnego przeglądania, sortowania oraz filtrowania artykułów naukowych. Dokument zawiera logo czasopisma oraz numer ISSN, które są stale widoczne w nagłówku. `react-dropzone` do intuicyjnej obsługi przesyłania plików metodą „przeciągnij i upuść”.
- **Stylizacja:** Warstwę wizualną zaimplementowano z użyciem CSS, który pozwala na tworzenie responsywnych, w tym nagłówka stale wyświetlającego logo czasopisma i numer ISSN.

2.3 Diagram klas UML



Rysunek 1: Diagram UML

Diagram przedstawia model klas systemu zarządzania artykułami. Klasa *Article* przechowuje dane dotyczące artykułu (m.in. tytuł, autorów, datę publikacji, pliki oraz liczbę wyświetleń) i umożliwia ich aktualizację poprzez metodę zwiększania liczby wyświetleń.

Klasa *Administrator* odpowiada za zarządzanie artykułami i udostępnia operacje ich dodawania, edycji, usuwania oraz przeglądania statystyk. Jeden administrator może zarządzać wieloma artykułami, natomiast każdy artykuł jest przypisany do jednego administratora.

Każdy artykuł posiada kategorię określoną przez typ wyliczeniowy *Category*. Hasła administratorów nie będą przechowywane w postaci jawnej - w systemie zostanie zastosowane ich bezpieczne hashowanie.

2.4 Logika aplikacji

- **Moduł artykułów:** Obsługuje pobieranie, filtrowanie i wyświetlanie artykułów.
- **Moduł użytkowników:** Zarządza kontami oraz rolami użytkowników.
- **Moduł publikacji:** Obsługuje dodawanie artykułów.
- **Repozytoria:** Warstwa dostępu do danych z wykorzystaniem Entity Framework Core.
- **Bezpieczeństwo:** Hasła przechowywane jako hashe, autoryzacja oparta na JWT.

2.5 Logowanie

Zgodnie z założeniami systemu, zwykły czytelnik nie posiada konta i nie loguje się do aplikacji. Moduł logowania przeznaczony jest wyłącznie dla administratora systemu, który dzięki temu uzyskuje dostęp do funkcji zarządzania treścią czasopisma.

Proces uwierzytelniania został zaprojektowany z naciskiem na bezpieczeństwo i opiera się na standardzie JWT (JSON Web Tokens). Takie podejście całkowicie wyeliminuje konieczność posiadania, generowania i instalowania lokalnych certyfikatów logowania przez administratora. Dodatkowo proces autoryzacji będzie chroniony mechanizmem uwierzytelniania wieloskładnikowego (MFA).

Aby zalogować się do systemu, administrator będzie musiał wykonać następujące kroki:

1. Przejść na dedykowaną podstronę logowania (np. `/admin/login`).
2. Wprowadzić swój identyfikator (login) oraz hasło w formularzu autoryzacyjnym.
3. Po wstępnej weryfikacji poświadczeń, system wymusi drugi krok weryfikacji (MFA), prosząc o podanie jednorazowego kodu (np. wygenerowanego w aplikacji typu Google Authenticator lub przesłanego na autoryzowany adres e-mail).

4. Po poprawnym wprowadzeniu kodu, serwer generuje i zwraca bezpieczny token `JWT`, który jest przechowywany w pamięci przeglądarki i służy do autoryzacji wszelkich późniejszych akcji w panelu administracyjnym (dodawanie, edycja, usuwanie publikacji).